

1. Methodischer Ansatz und Arbeitsstrategie

Für die Entwicklung meines Specialized Projects habe ich bewusst keinen rein theoretischen Ansatz gewählt, sondern eine Kombination aus:

- Iterativer Entwicklung
- Learning by Doing
- Praktischer Problemanalyse
- Kontinuierlicher Optimierung

Da ich dieses Projekt alleine umgesetzt habe, war es besonders wichtig, strukturiert vorzugehen und eigene Entscheidungen kritisch zu hinterfragen.

Zu Beginn analysierte ich bestehende Plattformen mit ähnlicher Funktionalität. Dabei untersuchte ich:

- Funktionsumfang
- Benutzerführung (UX)
- Design-Struktur
- Datenaufbau
- Monetarisierungsmodelle

Das Ziel war nicht, diese Plattformen zu kopieren, sondern ihre Stärken und Schwächen zu verstehen. Dadurch konnte ich bewusste Entscheidungen treffen und ein eigenständiges Konzept entwickeln. Gleichzeitig stellte ich mir eine zentrale strategische Frage: Welches konkrete Problem löst meine Plattform und warum ist sie sinnvoll? Diese Reflexion half mir, den Fokus während der gesamten Entwicklung nicht zu verlieren.

2. Konzeptentwicklung und Systemarchitektur

Bevor ich mit dem Programmieren begann, entwickelte ich das gesamte Systemkonzept auf Papier. Ich skizzierte:

- Den vollständigen User-Workflow
- Die Interaktion zwischen Nutzern
- Das Anfrage- und Bewertungssystem
- Die Struktur der Datenbank

Die Datenbankstruktur wurde mehrfach überarbeitet. Anfangs enthielt sie redundante Felder und unklare Beziehungen. Durch wiederholtes Zeichnen, Streichen und Neuplanen erreichte ich eine normalisierte Struktur mit klar definierten Relationen zwischen:

- User
- Skill
- Anfrage
- Bewertung

Diese Phase war entscheidend, da eine saubere Datenbankarchitektur die Grundlage für ein stabiles Backend bildet. Zusätzlich suchte ich nach einem geeigneten grafischen Tool zur Visualisierung der Datenbank, um Relationen besser darstellen und Denkfehler frühzeitig erkennen zu können.

3. Technische Lernphase und Strukturierungsprobleme

Ein großer Teil des Projekts bestand aus parallelem Lernen und Umsetzen. Während der Entwicklung vertiefte ich meine Kenntnisse in:

- Python
- Django
- Datenbankmodellen
- Template-Vererbung
- Formularverarbeitung

Zu Beginn machte ich typische Strukturierungsfehler. Beispielsweise legte ich alle Funktionen in einem einzigen Projektbereich ab. Später erkannte ich, dass Django eine modulare Struktur mit einzelnen Apps vorsieht. Nach dieser Erkenntnis restrukturierte ich das gesamte Projekt. Diese Entscheidung war zeitintensiv, führte jedoch zu:

- Besserer Wartbarkeit
- Klarer Trennung von Verantwortlichkeiten
- Sauberer Code-Struktur
- Höherer Professionalität

Mehrmals musste ich das Projekt vollständig neu aufsetzen. Diese Rückschritte waren frustrierend, jedoch ein wichtiger Bestandteil meines Lernprozesses. Eine zentrale Erkenntnis war: Eine gründliche Vorbereitung der Entwicklungsumgebung und der benötigten Tools verhindert spätere Systemkonflikte.

4. Frontend-Strategie und Designentscheidungen

Nach der Backend-Stabilisierung begann ich mit der Frontend-Umsetzung. Hier musste ich nicht nur technisch, sondern auch gestalterisch denken:

- Welche Farben repräsentieren die Plattform?
- Welche Schrift wirkt modern und lesbar?
- Wie kann ich ein konsistentes Layout erzeugen?
- Wie kann ich Wiederholungen im Code vermeiden?

Die Implementierung eines zentralen Base-Templates war ein wichtiger Schritt, um Template-Vererbung effizient zu nutzen. Ein großes Ziel war die vollständige Responsivität der Anwendung. Dabei lernte ich, dass responsives Design nicht nur technische Anpassung bedeutet, sondern auch strukturelles Umdenken im Layout.

5. Zentrale Probleme und deren Lösung (S3 – Problemlösungskompetenz)

Problem 1: Daten wurden nicht gespeichert

Ursachen:

- Fehlerhafte Formularbindung
- Falsche Model-Zuordnung
- Fehlende request.POST Verarbeitung

Lösung:

- Systematisches Debugging
- Analyse der Fehlermeldungen
- Vergleich mit Dokumentation
- Testweise Reduktion des Codes

Problem 2: Suchfunktion

Die Implementierung einer flexiblen Suchfunktion war technisch anspruchsvoll.

Herausforderung:

- Mehrere Filter kombinieren
- Performance berücksichtigen
- Leere Ergebnisse korrekt behandeln

Lösung:

- Optimierte QuerySets
- Bedingte Filterlogik
- Schrittweise Tests einzelner Parameter

Problem 3: Responsives Layout

Unterschiedliche Bildschirmgrößen führten zu Darstellungsfehlern.

Lösung:

- Flexible Grid-Strukturen
- Relative Größenangaben
- Wiederholtes Testen auf verschiedenen Geräten

Problem 4: Projektorganisation

Mehrfaches Neuaufsetzen des Projekts kostete Zeit.

Lösung:

- Bessere Planung der Ordnerstruktur
- Dokumentation der Schritte
- Klare Trennung von Entwicklungsphasen

6. Kritische Selbstbewertung (K2 – Bewertung eigener Arbeitsprozesse)

Rückblickend erkenne ich folgende Verbesserungspotentiale:

- Frühere Planung der Gesamtarchitektur
- Nutzung von Versionskontrolle von Beginn an
- Frühzeitige Integration von Tests
- Bessere Zeitpuffer für unerwartete Probleme
- Frühere Entscheidung für ein konsistentes Designsystem

Ich habe gelernt, dass technische Probleme selten isoliert auftreten, sondern oft durch unklare Planung entstehen.

7. Optimierungsmöglichkeiten für die Zukunft



Falls das Projekt weiterentwickelt wird, wären folgende Optimierungen sinnvoll:

8. Persönliche Entwicklung und Fazit

Dieses Projekt war eine intensive Lernphase. Neben technischen Fähigkeiten entwickelte ich:

- Durchhaltevermögen
- Analytisches Denken
- Problemlösungskompetenz
- Selbstorganisation
- Kritische Selbstreflexion

Mehrfaches Scheitern führte nicht zum Abbruch, sondern zu besseren Lösungen. Am Ende wurde mir bewusst: Nachhaltige Softwareentwicklung entsteht nicht durch Perfektion beim ersten Versuch, sondern durch kontinuierliches Verbessern. Dieses Projekt hat mir gezeigt, dass Fehler kein Zeichen von Schwäche sind, sondern ein notwendiger Teil professioneller Entwicklung.

